

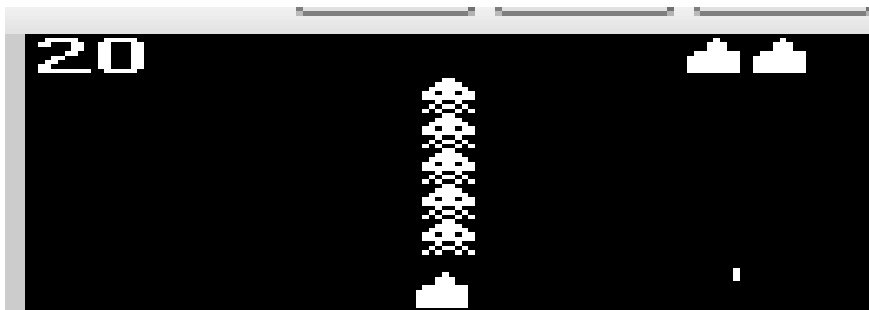
과제1 Q.1 과 Q.4 의 베릴로그 RTL에서 그래픽 장치를 다루는 방식의 차이점에 대하여 설명할 것

Q1에서는 vga_x, vga_y가 입력으로 들어오고, Verilog RTL은 받은 화면 좌표를 이용해 해당 위치에 우주선이나 외계인 등의 픽셀을 출력한다. 즉, 외부에서 주어진 좌표를 보고 그 위치에 무엇을 그릴지 결정하는 방식이다.

반면 Q4에서는 lcd_x, lcd_y와 픽셀 데이터를 RTL에서 출력하여 테스트벤치나 그래픽 장치에 전달한다. 테스트벤치는 이 데이터를 받아 실제 게임 화면처럼 표시한다.

따라서 Q1과 Q4는 단순히 그래픽을 생성하고 확인하는 차이뿐만 아니라, 그래픽 좌표가 RTL로 들어오는지 또는 RTL에서 나가는지에 따라 그래픽을 처리하는 방식 자체가 다르다.

과제2 10 의 실습을 수행 할 것. 스프라이트 시현 방식에 대하여 아두이노 C++와 베릴로그 RTL의 차이를 비교 설명 할 것.



Arduino C++와 Verilog RTL 모두 비트맵을 이용해 스프라이트를 표현한다는 점은 같다. 하지만 Arduino에서는 drawBitmap()과 같은 함수를 호출하면 라이브러리가 스프라이트의 픽셀들을 화면에 그려준다. 반면 Verilog RTL에서는 현재 스캔 중인 화면 좌표와 스프라이트 위치를 비교하고, 해당 비트맵 값이 1인지 직접 판단하여 픽셀을 출력한다. 따라서 Arduino는 함수 중심으로 그래픽을 처리하고, Verilog RTL은 픽셀 출력 조건을 직접 하드웨어 논리로 구성한다는 차이가 있다

즉 아두이노는 함수가 알아서 처리하고 베릴로그 rtl에서는 현재 픽셀 위치를 비트맵과 계속 대조하고 그 위치가 그려야하는 값이면 해당 픽셀을 출력한다.

과제3 Q.11 의 실습을 수행 할 것. 좌우 화살표 키보드 입력에 대하여 이상 반응을 보이는 이유를 찾을 것.

* Q.11의 SystemC 테스트 벤치가 변경된 점을 찾아볼 것.

* 키보드 입력을 보완하기 위해 베릴로그 RTL의 차이점을 찾아볼 것

```

STATE_PLAY: begin
    die_timer <= 6'b0;
    if(game_tick)begin
        if (btn_left && (cannon_x > 7'd1))
            cannon_x <= cannon_x - 1'b1;

        else if (btn_right && (cannon_x < (SCREEN_WIDTH - OBJ_WIDTH - 7'd2)))
            cannon_x <= cannon_x + 1'b1;
    end
end

```

실습에서는 좌우 화살표 키를 눌렀을 때 플레이어가 너무 빠르게 움직이는 문제가 있었다. 원인은 btn_left와 btn_right 입력이 시스템 클럭마다 계속 처리되어, 짧게 눌러도 cannon_x 값이 여러 번 바뀌기 때문이었다.

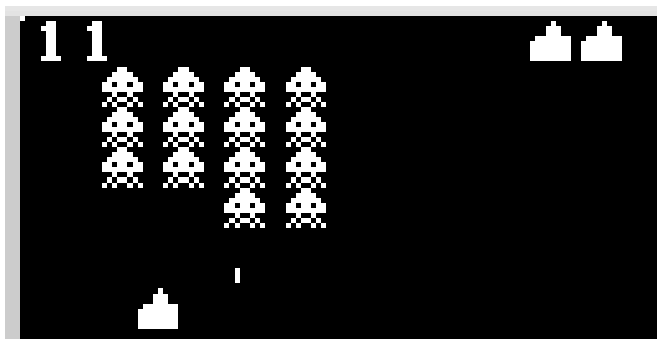
이를 해결하기 위해 좌우 이동 부분을 if(game_tick) 안에 넣어서, 매 클럭이 아니라 game_tick이 발생할 때만 이동하도록 수정하였다. 또한 왼쪽과 오른쪽 입력을 if와 else if로 구분하여 두 방향이 동시에 처리되지 않도록 하였다.

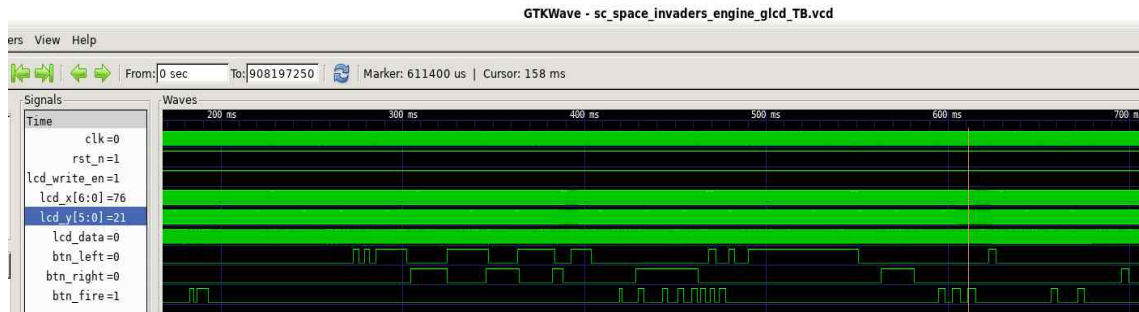
수정 후에는 키 입력에 대한 이동 속도가 전보다 많이느려지게 됐지만 플레이어가 이전보다 자연스럽게 좌우로 움직이게 되었다.

Q9와 Q11의 SystemC 테스트벤치는 Q9b와 비교했을 때 키보드 입력 처리 부분이 조금 보완되었다. Q9b에서는 SDL_GetKeyboardState()로 바로 키 상태를 읽었지만, Q11에서는 그 전에 SDL_PumpEvents()를 사용해서 키보드 이벤트 상태를 먼저 갱신하도록 바뀌었다. 또한 state != nullptr 조건을 추가해서 키보드 상태를 정상적으로 받아왔을 때만 btn_left, btn_right, btn_fire 신호를 전달하도록 하였다. 두 코드 모두 wait(16.6, SC_MS)를 사용하기 때문에 키보드 입력을 확인하는 주기 자체는 같다

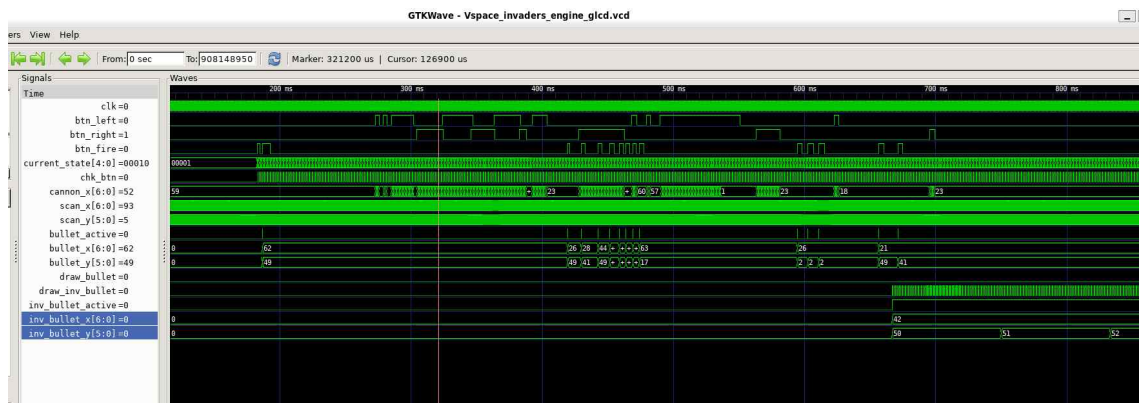
일반적인 소프트웨어에서는 키 입력을 한 번의 이벤트처럼 처리할 수 있지만, Verilog RTL에서는 입력 신호가 HIGH인 동안 매 클럭마다 계속 반응할 수 있다는 차이가 있다

과제4 Q.13의 시뮬레이션 디렉토리에서 실습을 실시할 것.





sc_space_invaders_engine_glcd_TB.vcd 파형



Vspice_invaders_engine_glcd.vcd 파형

make build 후 make wave를 통해 파형을 볼수있었고, make run을 통해 Q13의 시뮬레이션을 실행해 본 결과, 좌우 방향키 입력에 따라 플레이어가 이전보다 훨씬 부드럽고 자연스럽게 움직였다. Q11에서는 이동 속도가 너무 빠르거나 game_tick을 적용했을 때 다소 느려지는 문제가 있었지만, Q13에서는 키 입력과 플레이어 이동이 더 안정적으로 연결되어 조작감이 개선된 것을 확인할 수 있었다.

* 자신만의 문장으로 이 둘(소프트웨어 언어와 하드웨어 언어)을 비교할 수 있도록 정리하시기 바랍니다. 정리에 아래와 같은 관점을 포함 시키도록 하십시오.

소프트웨어 언어는 컴퓨터한테 “이거 하고, 그다음 이거 해”라고 순서대로 시키는 말이라고 보면 된다.

반대로 하드웨어 언어는 “이 회로는 지금 뭐를 하고 있고, 어떤 신호가 들어오면 어떻게 바뀌는지”를 설명하는 말이다.

하드웨어 코드를 실제 회로로 만들려면 RTL 방식으로 써야 한다. 쉽게 말하면 어떤 값을 저장하고, 클럭이 들어올 때 그 값이 어떻게 바뀌는지를 적는 것이다.

하드웨어를 표현할 때는 0, 1만 쓰는 게 아니라 값이 아직 정해지지 않은 X, 연결이 끊어진 것처럼 보이는 Z도 쓸 수 있다. 또 회로는 여러 부분이 동시에 움직이기 때문에 시간과 클럭도 중요하다. 그리고 신호가 0에서1로 바뀌는 순간 같은 특정 순간도 따로 볼 수 있다.

쉽게 정리하면, 소프트웨어는 “무슨 일을 순서대로 할지”를 적는 것, 하드웨어 언어는 “회로

가 동시에 어떻게 움직이고 신호가 언제 바뀌는지”를 적는 것이라고 보면 된다.

* SystemC에 대한 이해를 높이도록 하십시오. 인공지능이 알려준 장점을 나름대로 정리하십시오. SystemC는 C++ 언어로 하드웨어를 묘사하기 위해 앞서 언급한 하드웨어 개념을 갖춘 클래스 라이브러리이며 이를 시뮬레이션 하는 병렬 시뮬레이션 엔진을 포함하고 있습니다.

SystemC는 Verilog와 같은 언어는 아니고, C++을 이용해서 하드웨어의 동작을 표현할 수 있게 만든 라이브러리이다. 처음에는 C++ 코드가 Verilog 안에서 직접 동작하는 것이라고 생각했지만, 실제로는 Verilog 코드와 SystemC 코드가 따로 있고 시뮬레이션 과정에서 서로 연결된다. Verilog RTL은 실제 회로의 동작을 나타내고, SystemC는 그 회로에 입력을 주거나 출력 결과를 확인하는 테스트벤치로 사용할 수 있다.

SystemC는 C++ 기반이기 때문에 class와 sc_module을 사용해서 하드웨어 블록이나 테스트 벤치를 만들 수 있다. 하지만 클래스만 사용하는 것은 아니고, sc_signal로 신호를 주고받고, wait()를 이용해서 시간 흐름을 만들 수 있다. 또한 클럭의 상승이나 하강 같은 이벤트를 기준으로 동작하게 할 수 있고, 여러 동작이 동시에 진행되는 것도 시뮬레이션할 수 있다.

이번 실습에서는 SystemC가 클럭 생성, 리셋, 키보드 입력 전달, Verilog RTL과 신호 연결, 시뮬레이션 시간 관리 등에 사용되었다.

